Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **Информатика и системы управления**
КАФЕДРА **Компьютерные системы и сети (ИУ6)**
НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.01 Информатика и вычислительная техника**
БАКАЛАВРСКАЯ ПРОГРАММА **09.03.01/03 Вычислительные машины, комплексы, системы и сети**

ОТЧЕТ

по лабораторной работе № 2

Вариант 14

«Создание верификационного окружения для модуля mant_shifter»

Дисциплина: **Организация ЭВМ и систем**

Студент группы ИУ6-72Б _____ С.Ю. Кондратов
Подпись, дата *(И.О. Фамилия)*

Преподаватель _____ А.Ю. Попов
Подпись, дата *(И.О. Фамилия)*

Оценка _____

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 Тестируемое устройство.....4
1.1 Интерфейсы устройства.....4
1.2 Функциональность.....4
2 Тестовое окружение.....6
2.1 Интерфейс окружения.....6
2.2 Обертка тестируемого устройства.....6
2.3 Агенты окружения.....7
2.4 Транзакции и драйверы.....8
2.5 Мониторы.....9
3 Проверка окружения.....10
3.1 Временная диаграмма базового теста.....10
3.2 Результаты CI.....10
ЗАКЛЮЧЕНИЕ.....12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....13

ВВЕДЕНИЕ

Цель работы – создать на языке SystemVerilog [3] верификационное окружение для цифрового блока по индивидуальному варианту: интерфейс, обертку тестируемого устройства и агенты для всех его интерфейсов [1].

Задачи работы:

– изучить спецификацию тестируемого устройства и его интерфейсов;

– описать интерфейс окружения и обертку тестируемого устройства;

– разработать транзакции, драйверы и мониторы агентов входного и выходного интерфейсов данных и подключить агенты к окружению;

– добиться успешного прохождения CI первого этапа.

Задание варианта 14: тестируемое устройство – модуль сдвига и округления мантисс чисел с плавающей точкой mant_shifter [2] с параметром MANT_W = 52, то есть с шириной мантиссы формата двойной точности IEEE 754 [4]. Работа выполнена в отдельной ветке iu6_72_kondratov репозитория курса, код этапа отправлен коммитом с меткой [ci phase 1].

Отчет оформлен по ГОСТ 7.32-2017 [6].

1 Тестируемое устройство

1.1 Интерфейсы устройства

Модуль mant_shifter имеет вход тактового сигнала, вход синхронного сброса и два потоковых интерфейса данных с сигналом валидности – входной и выходной (таблица 1). Ширина входа сдвига равна $clog2(MANT_W); при MANT_W = 52 это 6 бит, поэтому величина сдвига принимает значения от 0 до 63, в том числе большие ширины мантиссы.

Таблица 1 – порты модуля mant_shifter при MANT_W = 52

Сигнал	Направление	Ширина	Описание
clk	вход	1	тактовый сигнал
resetn	вход	1	синхронный сброс, активный уровень 0
in_mant	вход	52	входная мантисса
in_shift	вход	6	число разрядов сдвига вправо
in_valid	вход	1	признак валидности входных данных
out_mant	выход	52	сдвинутая и округленная мантисса
out_valid	выход	1	признак валидности выходных данных

Оба интерфейса данных ведут себя как простейшие потоковые интерфейсы с сигнализацией [2]: входные in_mant и in_shift принимаются в такте, где in_valid = 1, а результат выставляется на out_mant вместе с out_valid = 1. Сигнала готовности нет, блок обязан принимать данные в каждом такте. Устройство конвейерное: результат появляется через два такта после входа.

1.2 Функциональность

Блок выполняет сдвиг мантиссы вправо и округление результата – операцию, которая нужна при сложении чисел с плавающей точкой для выравнивания порядков [2, 4]. Перед сдвигом к мантиссе слева дописывается неявная единица, а справа – три дополнительных разряда: G (Guard), R (Round) и S (Sticky). Выпадающие при сдвиге биты попадают сначала в G, затем в R, а бит S равен логическому ИЛИ всех битов, выпавших правее R.

После сдвига разряды G, R, S отбрасываются, а мантисса округляется к ближайшему значению [5]:

– при G = 0 – округление вниз, результат не меняется;

– при G = 1 и R = 1 или S = 1 – округление вверх, к результату прибавляется единица;

– при G = 1, R = 0, S = 0 значение находится ровно посередине, и используется округление к четному: единица прибавляется только при единичном младшем бите результата.

Для варианта 14 отдельно рассматриваются крайние случаи. При нулевом сдвиге мантисса проходит без изменений: неявная единица в разрядную сетку не попадает, а G = R = S = 0. При сдвигах 53–63 мантисса уходит за разрядную сетку целиком, неявная единица оказывается в G (сдвиг 53), в R (сдвиг 54) или в S, и результат равен 0 или 1. Если после сдвига мантисса состоит из одних единиц и округляется вверх, перенос из старшего разряда теряется и результат обнуляется.

2 Тестовое окружение

2.1 Интерфейс окружения

Интерфейс env_if включает все сигналы, выходящие из модуля, сгруппированные по входному и выходному интерфейсам данных (листинг 1). Используется тип logic, так как интерфейс подключается непосредственно к портам устройства и должен отображать значения x и z.

Листинг 1 – файл env_if.sv

```
//////////////////////////////////////////////////////////////////////////////////
   //////
// Company: Bauman Moscow State Technical University
// Engineer:
//
// Create date: 06/02/2025
// Project Name:
// Module Name:
// Target Devices:
// Tool Versions:
// Description: DUT interface.
//////////////////////////////////////////////////////////////////////////////////
   //////

interface env_if ();
  logic        clk;
  logic        resetn;

  // Входной интерфейс. MANT_W = 52, ширина сдвига $clog2(52) = 6 бит
  logic [51:0] in_mant;
  logic [5:0]  in_shift;
  logic        in_valid;

  // Выходной интерфейс
  logic [51:0] out_mant;
  logic        out_valid;

endinterface : env_if
```

2.2 Обертка тестируемого устройства

В обертке tb_dut_wrapper объявлен модуль mant_shifter с параметром MANT_W = 52, его порты подключены к сигналам интерфейса (листинг 2). В Makefile переменной DUT присвоено имя mant_shifter.

Листинг 2 – файл tb_dut_wrapper.sv

```
//////////////////////////////////////////////////////////////////////////////////
   //////
// Company: Bauman Moscow State Technical University
// Engineer:
//
// Create date: 11/02/2025
// Project Name:
// Module Name:
// Target Devices:
// Tool Versions:
// Description: DUT wrapper.
//////////////////////////////////////////////////////////////////////////////////
   //////

module tb_dut_wrapper (env_if env_intf);

  // DUT по варианту: mant_shifter, MANT_W = 52
  mant_shifter # (
    .MANT_W(52)
  )
  mant_shifter_inst (
    .clk      (env_intf.clk),
    .resetn   (env_intf.resetn),
    .in_mant  (env_intf.in_mant),
    .in_shift (env_intf.in_shift),
    .in_valid (env_intf.in_valid),
    .out_mant (env_intf.out_mant),
    .out_valid(env_intf.out_valid)
  );

endmodule : tb_dut_wrapper
```

2.3 Агенты окружения

Для каждого логически самостоятельного интерфейса устройства используется свой агент (таблица 2). Агенты тактового сигнала и сброса предоставлены в репозитории. Входной и выходной интерфейсы данных ведут себя одинаково, но имеют разный набор сигналов (на входе есть величина сдвига), поэтому для них созданы два разных агента: входной работает в активном режиме, выходной – в пассивном.

Таблица 2 – агенты тестового окружения

Агент	Режим	Сигналы	Назначение
clk_agent	активный	clk	генерация тактового сигнала с периодом 40 нс
resetn_agent	активный	resetn	выставление сброса и формирование транзакций начала и конца сброса
mant_in_agent	активный	in_mant, in_shift, in_valid	подача входных мантисс и сбор входных транзакций
mant_out_agent	пассивный	out_mant, out_valid	сбор выходных транзакций устройства

Каждый агент собран из транзакции, драйвера и монитора и унаследован от базового класса env_base_agent, секвенсор используется базовый. Классы агента собраны в собственный пакет (mant_in_agent_pkg, mant_out_agent_pkg), пакеты подключены к env_pkg, а каталоги агентов добавлены в files_tb.mk. В классе env агенты создаются в методе build(), получают ссылку на событие stop_event, режим is_active и виртуальный интерфейс и запускаются в методе run().

2.4 Транзакции и драйверы

Транзакция mant_in_tran содержит рандомизируемые поля mant (52 бита) и shift (6 бит) и служебное поле back_to_back, которое не относится к данным, а подсказывает драйверу, как подать транзакцию. Транзакция mant_out_tran содержит одно поле mant – выходную мантиссу. В обеих транзакциях переопределены методы compare() (сравнение полей данных), copy() (копирование полей из другой транзакции того же класса) и print() (вывод полей в шестнадцатеричном виде).

Драйвер mant_in_driver в методе init_signals() обнуляет in_mant и in_shift и снимает in_valid, так что после сброса на входе нет валидных данных. Метод drive_tran() по переднему фронту выставляет мантиссу, сдвиг и in_valid = 1. Если поле back_to_back сброшено, драйвер ждет следующий фронт, на котором устройство принимает данные, и возвращает входы в исходное состояние – получается разреженный поток. Если поле установлено, in_valid

не снимается и следующая транзакция выставляется уже на следующем такте – получается плотный поток, в котором данные идут каждый такт.

У выходного интерфейса нет сигнала готовности, поэтому драйвер mant_out_driver ничего не выставляет: метод drive_passive() не переопределен, а drive_tran() только выводит предупреждение о том, что агент должен работать в пассивном режиме.

2.5 Мониторы

Мониторы работают по заднему фронту тактового сигнала, когда сигналы уже установились, и не конфликтуют с драйвером, который работает по переднему фронту. Монитор mant_in_monitor в методе check_validity() считает такт валидным при in_valid = 1, а метод monitor_tran() собирает in_mant и in_shift в транзакцию mant_in_tran и отправляет ее в mon2out_mbx. Монитор mant_out_monitor аналогично собирает out_mant при out_valid = 1. Внутренних переменных у мониторов нет, поэтому метод reset() не переопределен; во время сброса базовый класс монитора транзакции не собирает.

3 Проверка окружения

3.1 Временная диаграмма базового теста

В скрипт add_signals.tcl добавлены сигналы интерфейса устройства: clk, resetn, in_mant, in_shift, in_valid, out_mant и out_valid. На рисунке 1 приведена временная диаграмма базового теста base_test, полученная моделированием окружения в Verilator [7] и снятая в GTKWave [8]. Агент тактового сигнала формирует clk с периодом 40 нс, агент сброса после нескольких тактов опускает resetn на пять тактов (с 220 до 420 нс), а драйвер входного агента в методе init_signals() держит in_valid = 0 и обнуляет шину данных. Валидных данных на входе нет, поэтому устройство не выставляет out_valid.

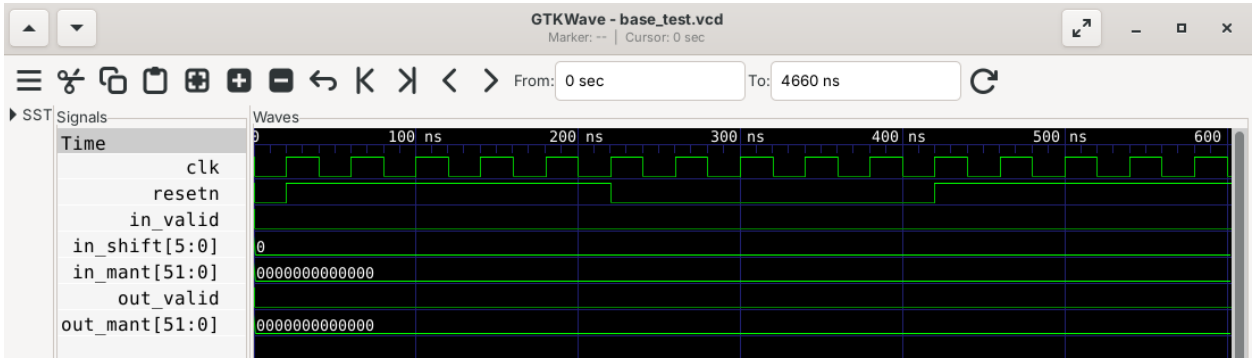


Рисунок 1 – временная диаграмма базового теста base_test

3.2 Результаты CI

Код этапа отправлен в ветку iu6_72_kondratov коммитом с меткой [ci phase 1]. Пайплайн завершился успешно: проверка файлов и компиляция прошли без ошибок, а базовый тест со встроенными проверками первого этапа завершился с результатом TEST PASSED (рисунок 2).

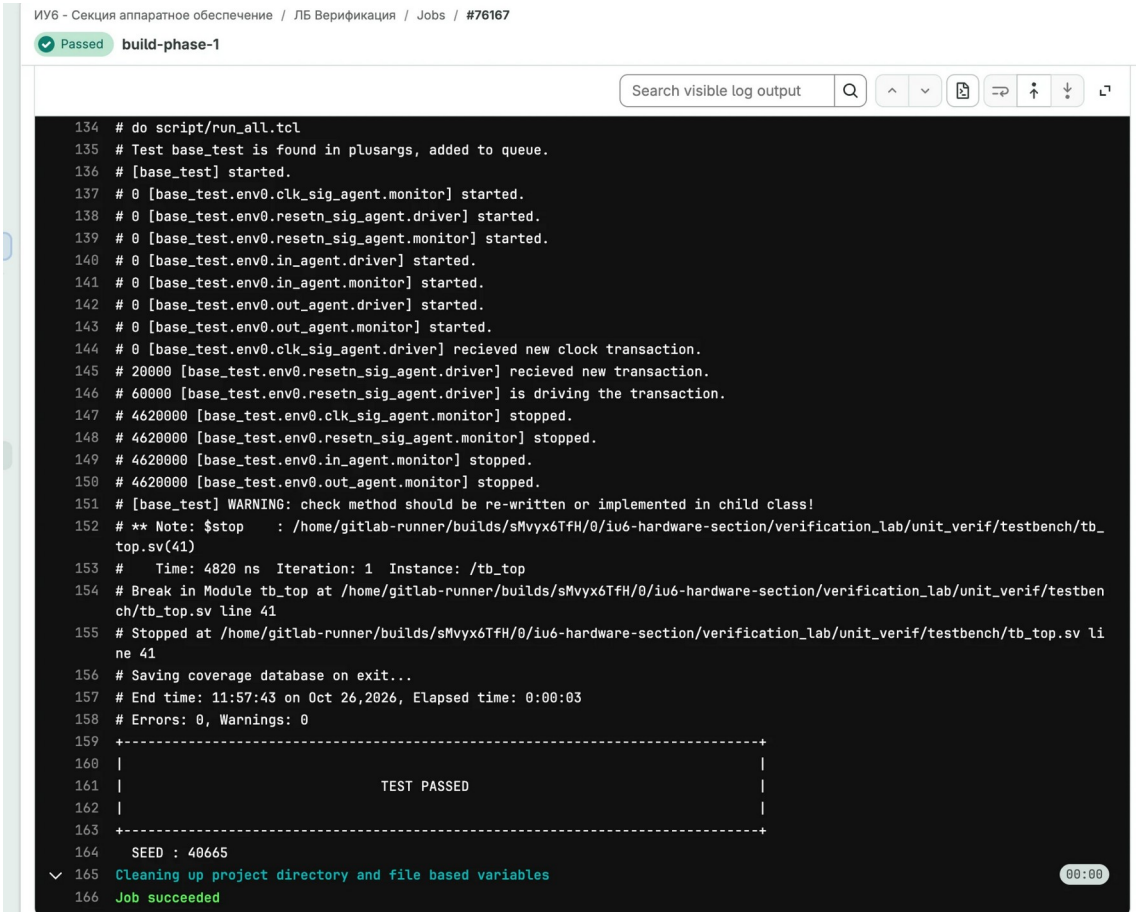


Рисунок 2 – лог джобы build-phase-1

ЗАКЛЮЧЕНИЕ

Для модуля сдвига и округления мантисс mant_shifter с MANT_W = 52 разработано верификационное окружение на SystemVerilog: интерфейс env_if, обертка tb_dut_wrapper, активный входной агент mant_in_agent и пассивный выходной агент mant_out_agent, которые работают вместе с предоставленными агентами тактового сигнала и сброса.

Драйвер входного агента формирует как разреженный, так и плотный поток данных, мониторы собирают транзакции по заднему фронту тактового сигнала и не конфликтуют с драйвером. Базовый тест подтверждает, что агенты собираются, запускаются и правильно работают с интерфейсом устройства, пайплайн CI первого этапа завершился успешно. Окружение готово к подключению эталонной модели и табло статистики на следующем этапе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лабораторные работы и ДЗ по верификации. Этапы № 1 – № 3: методические указания [Электронный ресурс] // Репозиторий verification_lab, МГТУ им. Н.Э. Баумана, кафедра ИУ6. – URL: https://gitlab.bmstu.ru/iu6-hardware-section/verification_lab (дата обращения: 26.10.2026).

2. Спецификация модуля сдвига и округления мантисс чисел с плавающей точкой mant_shifter [Электронный ресурс] // Репозиторий verification_lab. – URL: https://gitlab.bmstu.ru/iu6-hardware-section/verification_lab (дата обращения: 26.10.2026).

3. IEEE Std 1800-2017. IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language. – New York: IEEE, 2018. – 1315 p.

4. IEEE Std 754-2019. IEEE Standard for Floating-Point Arithmetic. – New York: IEEE, 2019. – 84 p.

5. Floating-point numbers and rounding [Электронный ресурс]. – URL: https://drilian.com/posts/2023.01.10-floating-point-numbers-and-rounding/ (дата обращения: 26.10.2026).

6. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017. – 27 с.

7. Verilator [Электронный ресурс]. – URL: https://verilator.org (дата обращения: 26.10.2026).

8. GTKWave [Электронный ресурс] : программа просмотра временных диаграмм. – URL: https://gtkwave.sourceforge.net (дата обращения: 26.10.2026).